Setting Up DVWA on Kali Linux in VMware

What is DVWA?

DVWA stands for **Damn Vulnerable Web Application**. It is a purposely vulnerable web application that was designed for security professionals, students, and developers to practice identifying and exploiting web vulnerabilities in a safe, controlled environment.

The goal of DVWA is to teach users about the various security issues that can arise in web applications, such as:

- **SQL Injection**: A type of attack where malicious SQL code is inserted into a query to manipulate a database.
- **Cross-Site Scripting (XSS)**: An attack that allows an attacker to inject malicious scripts into a web page.
- **Cross-Site Request Forgery (CSRF)**: A type of attack that tricks users into performing actions they didn’t intend to on a website.
- **File Upload Vulnerabilities**: Where attackers upload malicious files to a web server.

Security Levels in DVWA: DVWA offers three security levels (low, medium, and high), allowing users to practice exploiting vulnerabilities in environments with varying degrees of protection:

- **Low Security Level**: At this level, almost no input validation is performed, making all vulnerabilities easily exploitable. For example, in SQL Injection, the input fed into queries isn't filtered, allowing for malicious code to be injected with ease.

- **Medium Security Level**: At this level, some basic input validation is done to filter out malicious inputs, but it's not foolproof. This level allows users to understand how attacks bypass initial protections commonly found in many applications.

- **High Security Level**: At this level, the code is fully secured with proper input handling. Advanced input filtering techniques are used, and many attacks are prevented. This level simulates a highly secure environment, requiring more advanced techniques to find and exploit vulnerabilities.

These levels help users progressively practice and understand how to discover and fix security vulnerabilities in real-world applications.

It's an excellent tool for beginners to practice real-world hacking techniques in a legal and safe way without worrying about harming actual websites. However, it is important to always run DVWA in an isolated environment, like a virtual machine, to avoid security risks on your main computer or network.

Why Use DVWA?

- **Learn and Practice**: DVWA is perfect for hands-on practice with different web vulnerabilities.

- **Security Awareness**: It helps developers and security enthusiasts learn how to secure their applications by understanding common flaws.

- **Multiple Security Levels**: It provides low, medium, and high-security levels to give users experience with different levels of security challenges.

Step 1: Download DVWA

1. **Go to the web server folder:** The web server stores all the files that the browser can access. We need to go to the correct folder to place the DVWA files. In Linux, this is usually the `/var/www/html/` folder.

```
┌──(kali㉿kali)-[~]
└─$ cd /var/www/html/

┌──(kali㉿kali)-[/var/www/html]
└─$
```

```
cd /var/www/html/
```

2. **Download DVWA:** To get the DVWA files, we use a tool called **git** that downloads files from the internet. This command downloads DVWA to your server.

```
┌──(kali㉿kali)-[/var/www/html]
└─$ sudo git clone https://github.com/digininja/DVWA.git /var/www/html/dvwa
```

```
sudo git clone https://github.com/digininja/DVWA.git /var/www/html/dvwa
```

3. Check folder contents: Now, we check if the DVWA files are in the right place.

```
┌──(kali㉿kali)-[/var/www/html]
└─$ ls /var/www/html/dvwa/

CHANGELOG.md  README.ko.md  compose.yml  index.php         security.txt
COPYING.txt   README.md     config       instructions.php  setup.php
Dockerfile    README.pt.md  database     login.php         tests
README.ar.md  README.tr.md  docs         logout.php        vulnerabilities
README.es.md  README.vi.md  dvwa         php.ini
README.fa.md  README.zh.md  external     phpinfo.php
README.fr.md  SECURITY.md   favicon.ico  robots.txt
README.id.md  about.php     hackable     security.php
```

```
ls /var/www/html/dvwa/
```

Step 2: Set Permissions

1. **Change file permissions:** We need to give permission so that anyone using the server can read, write, and execute (run) the files inside the DVWA folder.

```
┌──(kali㉿kali)-[/var/www/html]
└─$ sudo chmod -R 755 /var/www/html/dvwa
```

```
sudo chmod -R 755 /var/www/html/dvwa
```

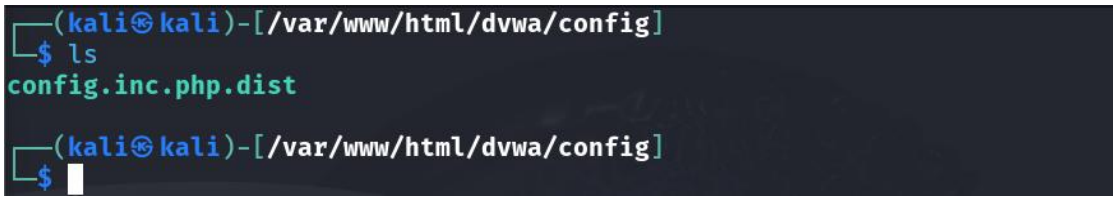
The command `sudo chmod -R 755 /var/www/html/dvwa` gives the file owner full control (read, write, execute), while allowing others to only read and execute the files, improving security compared to `777`.

Step 3: Configure DVWA

1. **Go to the DVWA configuration folder:** We need to change some settings in the configuration file so DVWA can connect to a database. First, let's go to the correct folder.

```
cd dvwa/config/
```

2. **Check folder contents**




```
ls
```

3. **Rename the configuration file:** There is a sample configuration file that we need to rename for DVWA to use.

```
┌──(kali㉿kali)-[/var/www/html/dvwa/config]
└─$ sudo mv config.inc.php.dist config.inc.php
```

```
sudo mv config.inc.php.dist config.inc.php
```

4. Edit the configuration file: We need to open the configuration file and make sure the settings for the database (where DVWA stores its data) are correct. To do this, we use a text editor called **nano**.

```
┌──(kali㉿kali)-[/var/www/html/dvwa/config]
└─$ sudo nano /var/www/html/dvwa/config/dvwa_config.php
```

```
sudo nano /var/www/html/dvwa/config/dvwa_config.php
```

```
  GNU nano 8.1        /var/www/html/dvwa/config/dvwa_config.php *
<?php
# Database management system to use
$DBMS = 'MySQL'; // Currently only MySQL is supported

# Database variables
$_DVWA = array();
$_DVWA['db_server']   = '127.0.0.1';      // MySQL server address
$_DVWA['db_database'] = 'dvwa';           // MySQL database name
$_DVWA['db_user']     = 'dvwa';           // MySQL username
$_DVWA['db_password'] = 'p@ssw0rd';       // MySQL password
$_DVWA['db_port']     = '3306';           // MySQL port (default is 3306)

# ReCAPTCHA settings (optional, you can leave these empty if you're not using>
$_DVWA['recaptcha_public_key']  = '';  // Your public reCAPTCHA key
$_DVWA['recaptcha_private_key'] = '';  // Your private reCAPTCHA key

# Default security level
$_DVWA['default_security_level'] = 'low';  // Default security level (low/med>

?>

^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify
```

```php
<?php

# Database management system to use
$DBMS = 'MySQL'; // Currently only MySQL is supported

# Database variables
$_DVWA = array();
$_DVWA['db_server']   = '127.0.0.1';      // MySQL server address
$_DVWA['db_database'] = 'dvwa';           // MySQL database name
$_DVWA['db_user']     = 'dvwa';           // MySQL username
$_DVWA['db_password'] = 'p@ssw0rd';       // MySQL password
$_DVWA['db_port']     = '3306';           // MySQL port (default is 3306)

# ReCAPTCHA settings (optional, you can leave these empty if you're not using
ReCAPTCHA)
$_DVWA['recaptcha_public_key']  = '';  // Your public reCAPTCHA key
$_DVWA['recaptcha_private_key'] = '';  // Your private reCAPTCHA key

# Default security level
$_DVWA['default_security_level'] = 'low';  // Default security level
(low/medium/high)

?>
```

5. **Exit MySQL**:

```
MariaDB [(none)]> exit
Bye

┌──(kali㉿kali)-[/var/www/html/dvwa/config]
└─$
```

```
exit
```

Step 4: Set Up MySQL Database

1. **Start MySQL:** DVWA needs a database to store its data. We start the MySQL service, which manages the database.

```
┌──(kali㉿kali)-[/var/www/html/dvwa/config]
└─$ sudo service mysql start
```

```
sudo service mysql start
```

2. Log in to MySQL: We log in to MySQL as the root (admin) user so we can set up the DVWA database.

```
┌──(kali㉿kali)-[/var/www/html/dvwa/config]
└─$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 45
Server version: 11.4.3-MariaDB-1 Debian n/a

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Support MariaDB developers by giving a star at https://github.com/MariaDB/serv
er
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

```
sudo mysql -u root -p
```

You will be asked for your MySQL root password.

3. Create the DVWA database : If the DVWA database doesn't already exist, we create it with this command:

```
MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.001 sec)
```

```
CREATE DATABASE dvwa;
```

We can cheack the list of databases: Once you are inside the MySQL interface by running the following command to list all databases:

```
SHOW DATABASES;
```

```
MariaDB [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| dvwa               |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.000 sec)

MariaDB [(none)]>
```

4. **View the users:** list of users in the system:

Press enter or click to view image in full size

```
MariaDB [(none)]> SELECT User, Host FROM mysql.user;
+-------------+-----------+
| User        | Host      |
+-------------+-----------+
| mariadb.sys | localhost |
| mysql       | localhost |
| root        | localhost |
+-------------+-----------+
3 rows in set (0.002 sec)

MariaDB [(none)]>
```

```
SELECT User, Host FROM mysql.user;
```

5. Create a new user and grant permissions : We create a user for DVWA and give it access to the database.

```
MariaDB [(none)]> CREATE USER 'dvwa'@'127.0.0.1' IDENTIFIED BY 'p@ssw0rd';
ERROR 1396 (HY000): Operation CREATE USER failed for 'dvwa'@'127.0.0.1'
MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'127.0.0.1';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]>
```

```
CREATE USER 'dvwa'@'127.0.0.1' IDENTIFIED BY 'p@ssw0rd';
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'127.0.0.1';
```

Step 5: Configure Apache and PHP

1. **Navigate to the correct PHP configuration file:** Based on the image, you're already in the `/etc/php/8.2/apache2/` directory. This is where the PHP configuration file for Apache is located.

```
┌──(kali㉿kali)-[/var/www/html/dvwa/config]
└─$ cd /etc/php/8.2/apache2/

┌──(kali㉿kali)-[/etc/php/8.2/apache2]
└─$
```

```
cd /etc/php/8.2/apache2/
```

2. **Open the PHP configuration file:** Use `nano` to open the `php.ini` file and configure the PHP settings. The `php.ini` file contains various settings for PHP and Apache.

```
┌──(kali㉿kali)-[/etc/php/8.2/apache2]
└─$ sudo nano /etc/php/8.2/apache2/php.ini
```

```
sudo nano /etc/php/8.2/apache2/php.ini
```

3. **Look for specific settings to configure:** Some common settings you might need to adjust:

- **allow_url_include**: Enable or disable the inclusion of files over the internet. This is sometimes required for web applications like DVWA

```
  GNU nano 8.1                /etc/php/8.2/apache2/php.ini

; Maximum number of files that can be uploaded via a single request
max_file_uploads = 20

;;;;;;;;;;;;;;;;;;;
; Fopen wrappers ;
;;;;;;;;;;;;;;;;;;;

; Whether to allow the treatment of URLs (like http:// or ftp://) as files.
; https://php.net/allow-url-fopen
allow_url_fopen = On

; Whether to allow include/require to open URLs (like https:// or ftp://) as >
; https://php.net/allow-url-include
allow_url_include = Off

; Define the anonymous ftp password (your email address). PHP's default setti>
; for this is empty.
; https://php.net/from
;from="john@doe.com"

; Define the User-Agent string. PHP's default setting for this is empty.
; https://php.net/user-agent
;user_agent="PHP"

; Default timeout for socket based streams (seconds)
; https://php.net/default-socket-timeout
default_socket_timeout = 60

^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify
```

```
; https://php.net/allow-url-include
allow_url_include = On
```

```
allow_url_include = On
```

memory_limit: Adjust the memory limit for PHP scripts, if needed.

```
; Maximum amount of memory a script may consume
; https://php.net/memory-limit
memory_limit = 128M
```

```
memory_limit = 128M
```

max_execution_time: The maximum amount of time a PHP script is allowed to run.

```
; https://php.net/max-execution-time
; Note: This directive is hardcoded to 0 for the CLI SAPI
max_execution_time = 30
```

```
max_execution_time = 30
```

4. **Save the changes and exit:**

- After making changes, press `CTRL + O` to save the file, then `Enter` to confirm.
- Press `CTRL + X` to exit the nano editor.

5. Restart Apache to apply changes: Once you've updated the PHP settings, restart the Apache service to apply the changes.

```
sudo service apache2 start
```

Check the Loaded PHP Version

To verify that PHP 8.2 is correctly configured and loaded by Apache, you can create a `phpinfo` file:

1. **Create the file in your web server directory:**

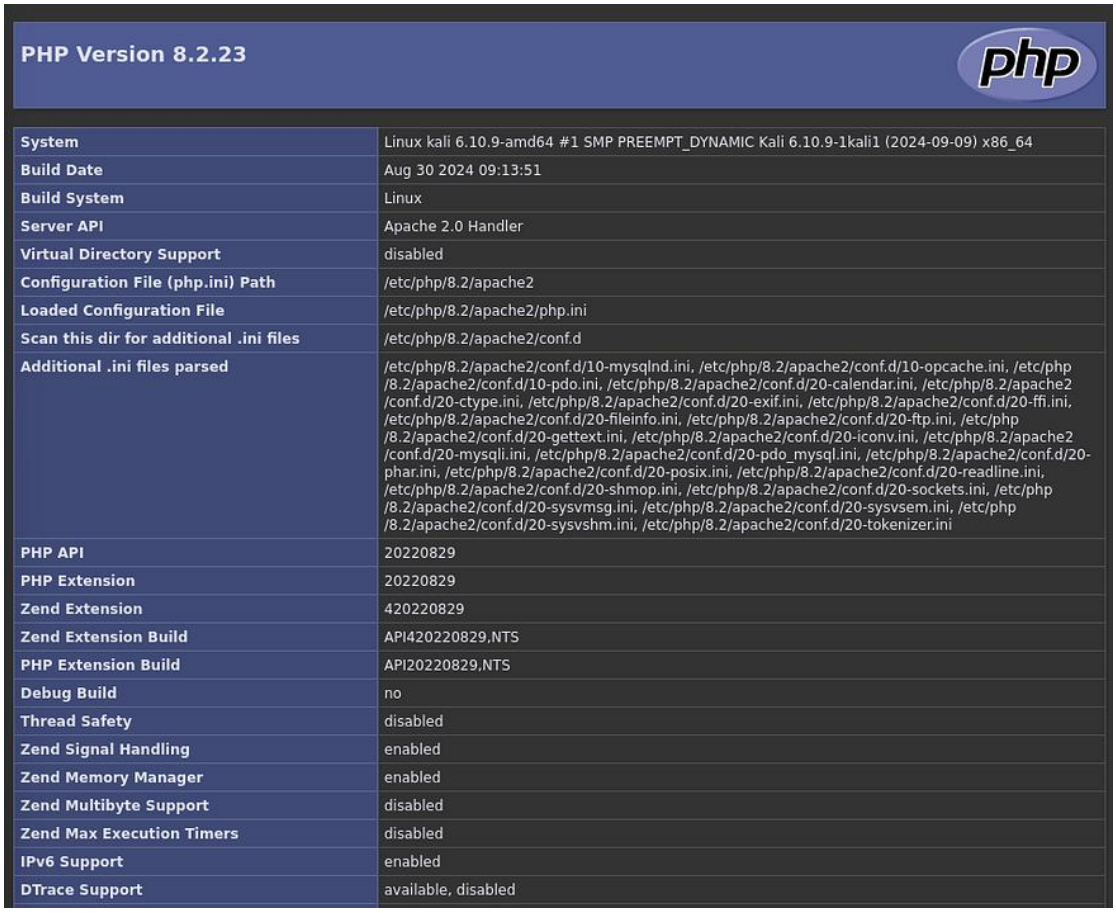
```
sudo nano /var/www/html/phpinfo.php
```

2. Add the following content:

```
<?php
phpinfo();
?>
```

3. **Checking in the Browser:** After saving the file and exiting nano, open your browser and navigate to:

```
http://127.0.0.1/phpinfo.php
```



PHP Version 8.2.23

System	Linux kali 6.10.9-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.10.9-1kali1 (2024-09-09) x86_64
Build Date	Aug 30 2024 09:13:51
Build System	Linux
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.2/apache2
Loaded Configuration File	/etc/php/8.2/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.2/apache2/conf.d
Additional .ini files parsed	/etc/php/8.2/apache2/conf.d/10-mysqlnd.ini, /etc/php/8.2/apache2/conf.d/10-opcache.ini, /etc/php/8.2/apache2/conf.d/10-pdo.ini, /etc/php/8.2/apache2/conf.d/20-calendar.ini, /etc/php/8.2/apache2/conf.d/20-ctype.ini, /etc/php/8.2/apache2/conf.d/20-exif.ini, /etc/php/8.2/apache2/conf.d/20-ffi.ini, /etc/php/8.2/apache2/conf.d/20-fileinfo.ini, /etc/php/8.2/apache2/conf.d/20-ftp.ini, /etc/php/8.2/apache2/conf.d/20-gettext.ini, /etc/php/8.2/apache2/conf.d/20-iconv.ini, /etc/php/8.2/apache2/conf.d/20-mysqli.ini, /etc/php/8.2/apache2/conf.d/20-pdo_mysql.ini, /etc/php/8.2/apache2/conf.d/20-phar.ini, /etc/php/8.2/apache2/conf.d/20-posix.ini, /etc/php/8.2/apache2/conf.d/20-readline.ini, /etc/php/8.2/apache2/conf.d/20-shmop.ini, /etc/php/8.2/apache2/conf.d/20-sockets.ini, /etc/php/8.2/apache2/conf.d/20-sysvmsg.ini, /etc/php/8.2/apache2/conf.d/20-sysvsem.ini, /etc/php/8.2/apache2/conf.d/20-sysvshm.ini, /etc/php/8.2/apache2/conf.d/20-tokenizer.ini
PHP API	20220829
PHP Extension	20220829
Zend Extension	420220829
Zend Extension Build	API420220829,NTS
PHP Extension Build	API20220829,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	disabled
Zend Max Execution Timers	disabled
IPv6 Support	enabled
DTrace Support	available, disabled

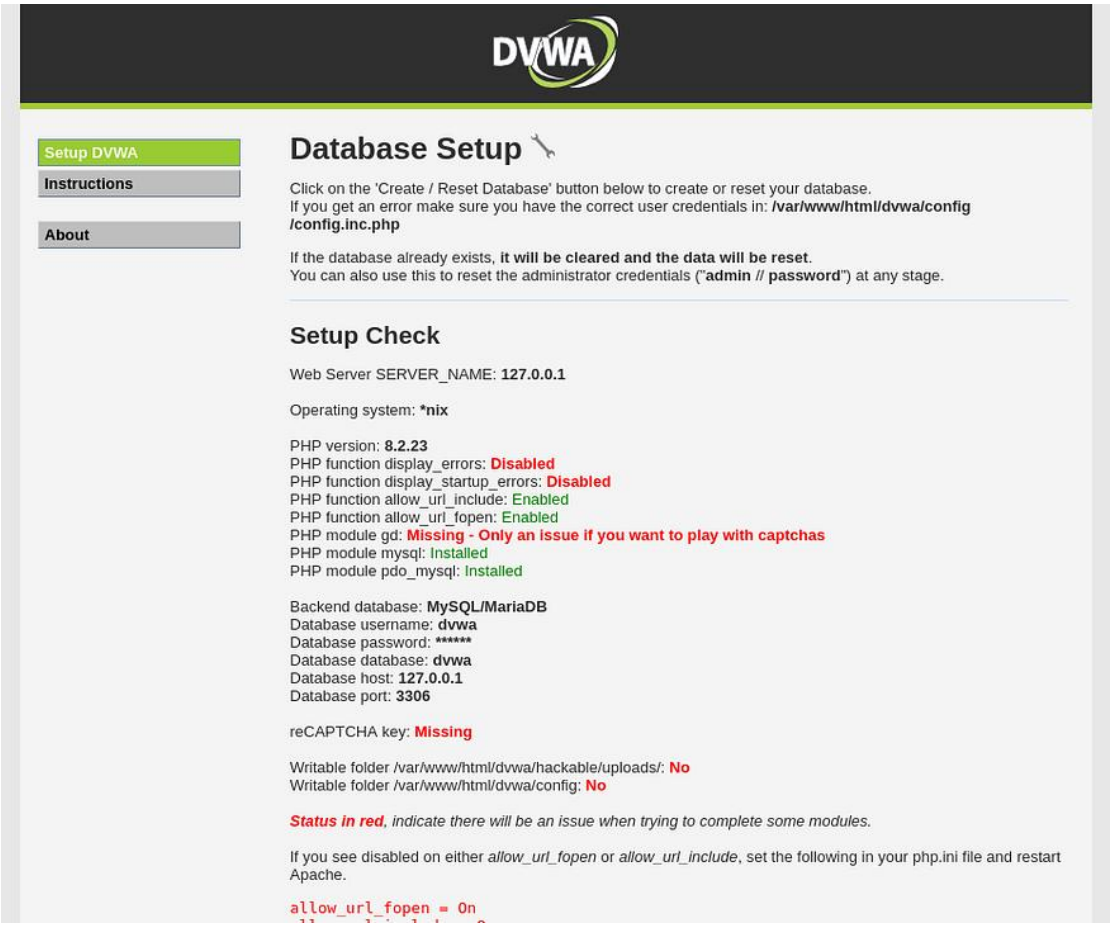
Th `phpinfo()` page loaded and it confirms that **PHP version 8.2.23** is running with Apache on your Kali Linux system.

Step 6: Try a CSRF Attack

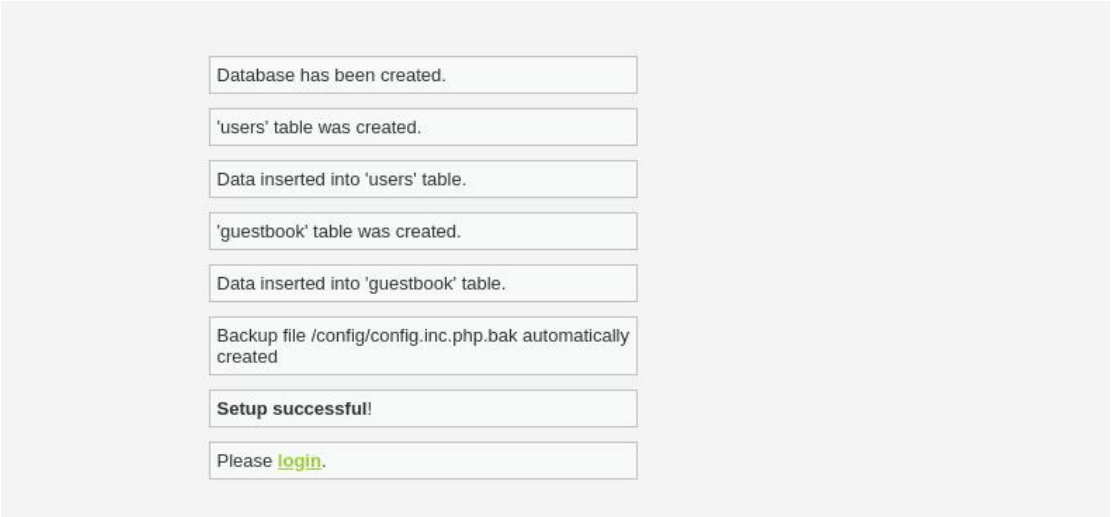
Now that DVWA is installed, we can try a simple CSRF attack.

1. **Open DVWA in the browser:** Open your web browser and type this address to open DVWA:

```
http://127.0.0.1/dvwa/setup.php
```



Click on Create/Reset Database



Login and go to the CSRF module: After logging in, go to the CSRF vulnerability section in the menu.

Use the default credentials below to log in.

Username: admin
Password: password

DVWA

- Home
- Instructions
- Setup / Reset DB

- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography

- DVWA Security
- PHP Info
- About

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with **various levels of difficultly**, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any Internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as VirtualBox or VMware), which is set to NAT networking mode. Inside a guest machine, you can download and install XAMPP for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources